Generación de imágenes cartoon con MPI y OpenMP

Práctico Final Integrador de Computación Avanzada · UADER · 2026

Resultados medidos el 28 de octubre de 2026 en una computadora Windows

1 Introducción y objetivos

El objetivo fue construir una imagen cartoon a partir de una imagen RGB y comparar una implementación secuencial con tres formas de paralelismo: MPI, OpenMP y una combinación de ambas. El análisis utiliza exclusivamente las ejecuciones conservadas en los CSV del proyecto. Cada salida paralela se comparó con la secuencial para el mismo tamaño, filtro y cantidad de niveles; la equivalencia binaria se confirmó en los doce casos exigidos por la consigna (UADER, 2026).

2 Fundamentos del procesamiento de imágenes

Una imagen RGB de 8 bits almacena tres canales por píxel, cada uno entre 0 y 255. Para detectar contornos se calcula una intensidad gris como 0,30R + 0,59G + 0,11B. El promedio espacial de una vecindad atenúa variaciones locales antes de aplicar Sobel, que estima cambios horizontales y verticales. La posterización reduce el número de valores posibles por canal y simplifica las regiones de color (Guerrero, 2017).

3 Algoritmo cartoon

El programa convierte la imagen a gris, aplica un promedio uniforme de 3 × 3 o 5 × 5 píxeles, calcula |Sx| + |Sy| con Sobel y marca un borde cuando el resultado supera el umbral 150. En paralelo lógico, cada canal RGB se cuantiza en 3 o 9 franjas, usando el punto medio de cada franja. La salida suma la máscara de bordes a la imagen posterizada y limita cada canal a 255. La opción de bordes oscuros resta esa máscara y limita inferiormente a cero; los benchmarks emplearon bordes claros.

4 Implementación secuencial

La referencia recorre todos los píxeles en un proceso. Las etapas producen vectores separados de gris, imagen borrosa y bordes; luego se posteriza una copia RGB y se combinan los resultados. El cronómetro steady_clock comienza después de leer el PPM y termina antes de escribir la salida. Esta medición define la base para cada speedup.

5 Implementación MPI

El proceso raíz lee el PPM y difunde ancho y alto con MPI_Bcast. Las filas se reparten en bloques contiguos, asignando las filas sobrantes a los primeros procesos. MPI_Scatterv envía bloques de tamaño variable con halos, y MPI_Gatherv reconstruye la imagen con las filas propias de cada proceso. La medición con MPI_Wtime comienza antes del reparto y termina después de la recolección, por lo que incluye esas comunicaciones. El código informa error si la lectura, la configuración o la escritura falla.

6 Implementación OpenMP

Las etapas de gris, borroneo, Sobel, posterización y combinación emplean parallel for. Cada iteración escribe una posición distinta; los acumuladores del promedio son variables locales del bucle. La barrera implícita al finalizar cada región asegura que la etapa siguiente lea una matriz completa. Se deshabilitó el ajuste dinámico de hilos y se informó el número efectivo. Se comprobaron 1, 2, 4 y 8 hilos.

7 Implementación híbrida MPI y OpenMP

Cada proceso MPI recibe un bloque con halos y ejecuta las etapas de píxeles con hilos OpenMP. Las llamadas MPI quedan fuera de las regiones paralelas. Se midieron 2 procesos × 2 hilos, 2 × 4 y 4 × 2. El programa informa el mínimo y máximo de hilos efectivos observados entre procesos; la prueba de casos límite verificó que coincidieran con la configuración solicitada.

8 División del problema

El borroneo de tamaño k necesita k/2 filas vecinas a cada lado, mientras que Sobel necesita una fila más sobre el resultado borroso. Por ello se distribuye un halo de k/2 + 1 filas: dos para 3 × 3 y tres para 5 × 5. Solo las filas propias vuelven al proceso raíz. El reparto contiguo simplifica las transferencias, aunque agrega cálculo redundante en los halos y copias de memoria.

9 Comunicación y sincronización

MPI_Barrier alinea el inicio de la región cronometrada y las operaciones colectivas coordinan reparto y recolección. No hay intercambio entre procesos durante el cálculo porque cada uno recibió el halo completo. Las barreras implícitas de OpenMP separan etapas dependientes. La comparación byte por byte de todas las salidas frente a la referencia ayuda a detectar errores en fronteras de bloques y carreras visibles en el resultado.

10 Metodología y equipo de pruebas

Se generaron escenas RGB deterministas de 800 × 800, 2000 × 2000 y 5000 × 5000 píxeles. Para cada tamaño se probaron filtros de 3 × 3 y 5 × 5 y posterización de 3 y 9 niveles: doce casos. Cada caso se ejecutó tres veces en once configuraciones, para un total de 396 mediciones. Los tiempos son los impresos por cada programa y excluyen lectura y escritura. El equipo utilizado tiene un AMD Ryzen 7 5700 y 16 procesadores lógicos; ejecutó Windows build 26200, GCC 16.2 y Microsoft MPI 10.1.3. Las pruebas se hicieron en un solo equipo, de modo que MPI mide comunicación entre procesos locales. La auditoría de CSV verificó tres repeticiones únicas por configuración, tiempos positivos, promedios, fórmulas y un único SHA-256 de salida por cada uno de los doce casos.

11 Resultados de rendimiento

OpenMP con 8 hilos obtuvo el menor promedio en 8 de los doce casos; la configuración híbrida 4 × 2 lo obtuvo en 4. La tabla 1 muestra la configuración ganadora de cada caso. El archivo resumen.csv conserva las 132 filas con procesos, hilos, tiempo, speedup y eficiencia; mediciones.csv conserva las 396 observaciones individuales.

Imagen	Filtro	Niv.	Versión	Proc.	Hilos	Tiempo s	Speedup	Efic.
800x800	3×3	3	OpenMP	1	8	0.0057	1.86	0.23
800x800	3×3	9	Híbrida	4	2	0.0044	2.50	0.31
800x800	5×5	3	Híbrida	4	2	0.0053	2.89	0.36
800x800	5×5	9	OpenMP	1	8	0.0053	2.96	0.37
2000x2000	3×3	3	OpenMP	1	8	0.0243	2.56	0.32
2000x2000	3×3	9	OpenMP	1	8	0.0257	2.71	0.34
2000x2000	5×5	3	OpenMP	1	8	0.0330	3.27	0.41
2000x2000	5×5	9	Híbrida	4	2	0.0306	3.31	0.41
5000x5000	3×3	3	OpenMP	1	8	0.1207	3.35	0.42
5000x5000	3×3	9	OpenMP	1	8	0.1227	3.26	0.41
5000x5000	5×5	3	OpenMP	1	8	0.1710	3.65	0.46
5000x5000	5×5	9	Híbrida	4	2	0.1729	3.85	0.48

Tabla 1. Configuración con menor tiempo promedio en cada caso. Speedup y eficiencia corresponden a esa misma configuración.

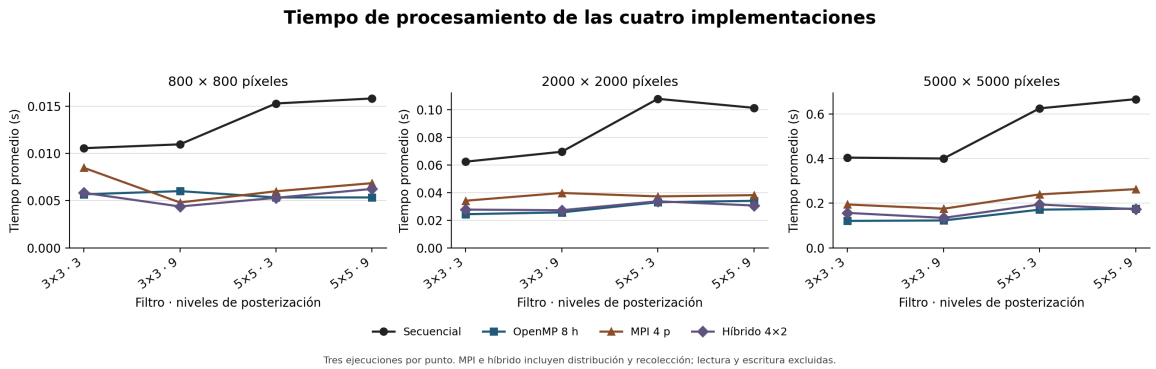


Figura 1. Tiempo de las cuatro implementaciones en configuraciones representativas.

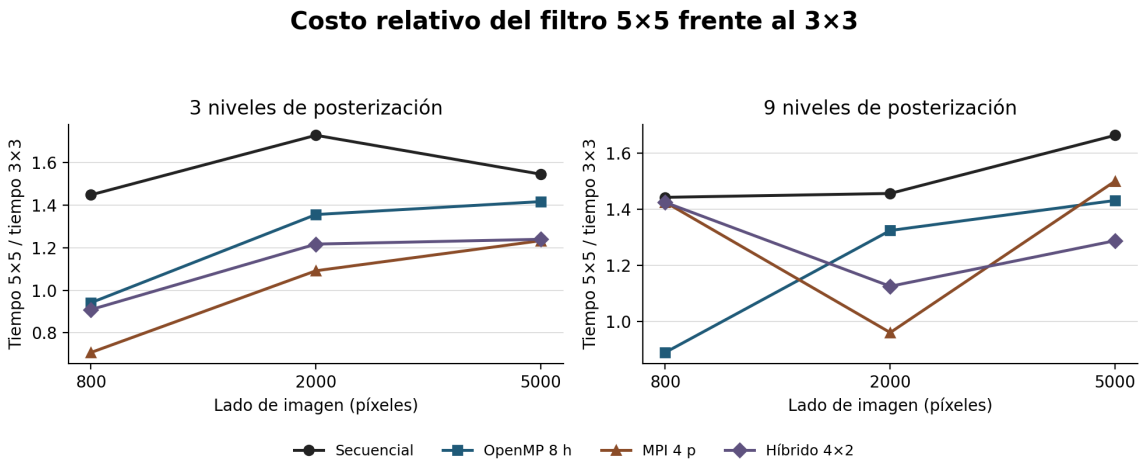


Figura 2. Relación entre los tiempos de filtros 5 × 5 y 3 × 3.

El filtro de 5 × 5 exige hasta 25 contribuciones por píxel, frente a 9 del filtro 3 × 3. En la versión secuencial, la relación media tiempo 5 × 5 / tiempo 3 × 3, promediada sobre los dos niveles, fue 1,45 en 800 × 800, 1,59 en 2000 × 2000 y 1,61 en 5000 × 5000. En algunas configuraciones paralelas pequeñas la relación cambia por variación de ejecución y sobrecarga, por lo que el costo del filtro no debe inferirse de un solo punto.

12 Speedup y eficiencia

Para cada caso se calculó speedup = promedio secuencial / promedio paralelo de la misma imagen, filtro y niveles. La eficiencia divide ese speedup por hilos para OpenMP, procesos para MPI o procesos × hilos para la híbrida. La tabla 2 resume la media de los cuatro speedups de filtro y niveles para cada tamaño. Son medias descriptivas de razones por caso, no el cociente entre tiempos agregados.

Imagen	OpenMP 8 h	MPI 4 p	Híbrida 2×4	Híbrida 4×2
800×800	2.38	2.10	2.17	2.43
2000×2000	2.88	2.28	2.52	2.83
5000×5000	3.51	2.38	2.94	3.16

Tabla 2. Speedup medio (veces) sobre las cuatro combinaciones de filtro y niveles de cada tamaño.

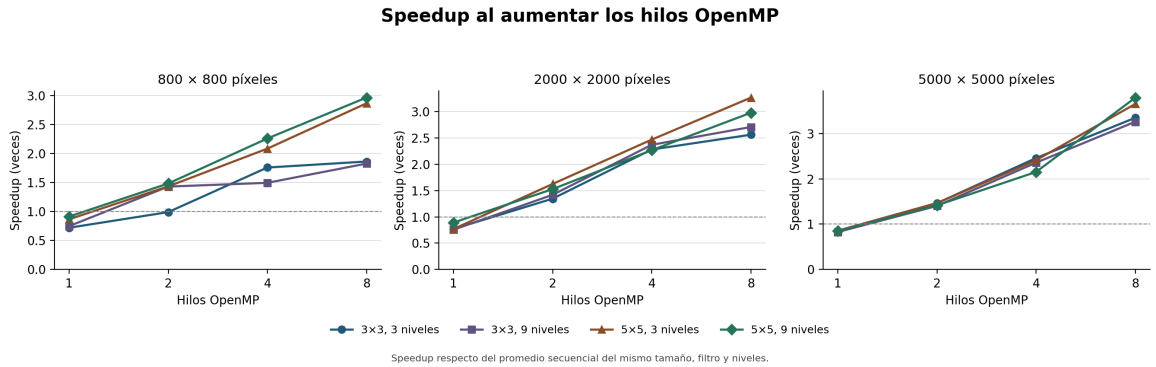


Figura 3. Escalamiento de OpenMP entre 1 y 8 hilos.

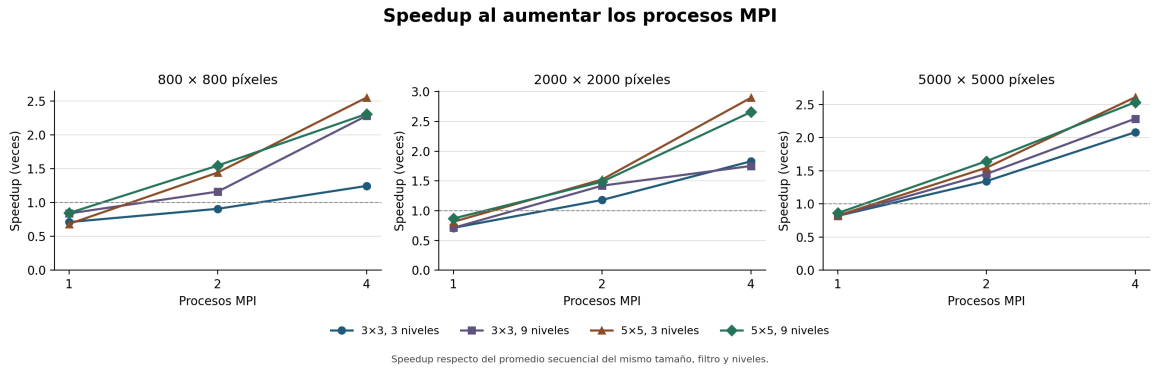


Figura 4. Escalamiento de MPI entre 1 y 4 procesos.

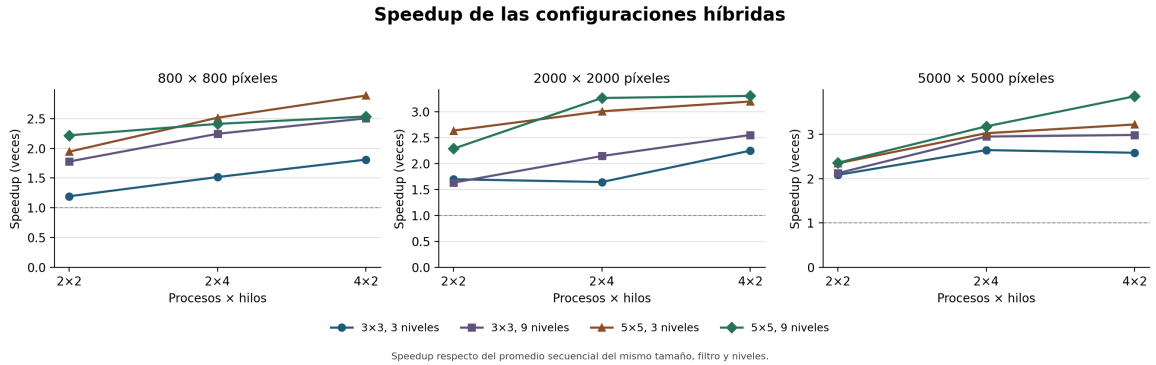


Figura 5. Comparación de tres configuraciones híbridas.

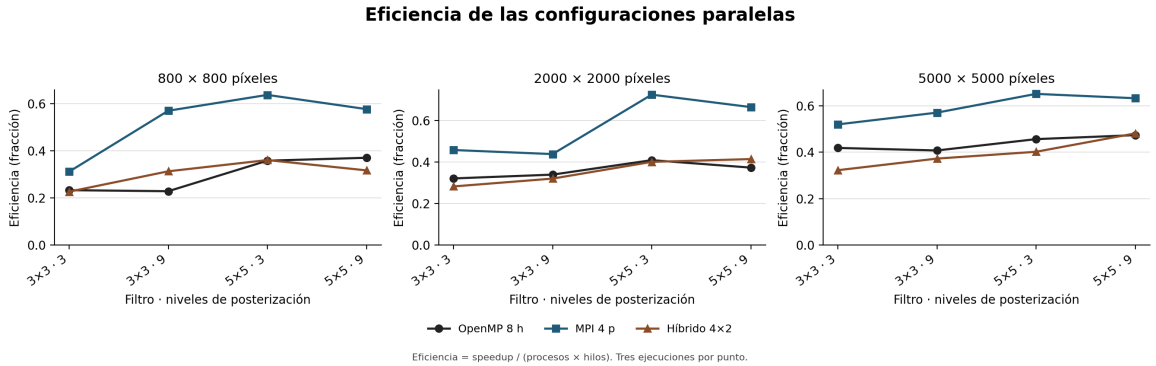


Figura 6. Eficiencia de configuraciones paralelas representativas.

En 5000 × 5000, el speedup medio de OpenMP con 8 hilos fue 3,51 y el de MPI con 4 procesos fue 2,38. Las dos configuraciones híbridas de ocho unidades obtuvieron 2,94 (2 × 4) y 3,16 (4 × 2). La eficiencia correspondiente a OpenMP de 8 hilos fue aproximadamente 0,44, frente a 0,39 para la híbrida 4 × 2, al promediar los cuatro casos. El aumento de unidades mejora el tiempo, pero con rendimiento marginal decreciente.

13 Comparación de implementaciones

OpenMP fue la opción más rápida en la mayoría de los casos medidos en esta máquina compartida. MPI escaló al pasar de uno a cuatro procesos, aunque su ejecución de un solo proceso fue más lenta que la referencia; además del cómputo, su tiempo incluye Scatterv, Gatherv y las copias de bloques. Este resultado es compatible con un costo de comunicación, pero el ensayo no separa ese costo de las otras diferencias de implementación. La híbrida 4 × 2 ganó cuatro casos y superó a la 2 × 4 en la media de speedup de 5000 × 5000. No se extrapolan estas comparaciones a un clúster ni a otros procesadores.

El promedio de speedup de OpenMP con 8 hilos creció de 2,38 en 800 × 800 a 2,88 en 2000 × 2000 y 3,51 en 5000 × 5000. El de MPI con 4 procesos pasó de 2,10 a 2,28 y 2,38. El aumento del trabajo por ejecución amortiza una parte de los costos fijos de iniciar regiones paralelas y mover datos, aunque el código no mide cada componente por separado. Con un único hilo, OpenMP alcanzó speedups medios de 0,81, 0,80 y 0,84 según el tamaño; con un solo proceso, MPI obtuvo 0,77, 0,78 y 0,83. Esas variantes ejecutan trabajo adicional y por ello no son equivalentes en costo a la referencia secuencial.

La distribución MPI de este programa se hace desde el proceso raíz mediante Scatterv de bloques ya ampliados; no se realiza un intercambio directo de halos entre vecinos durante el cálculo. Esta decisión simplifica el control de fronteras, pero agrega bytes enviados y trabajo redundante a medida que crece el número de procesos. La alternativa híbrida reduce el tamaño del bloque por proceso y lo divide entre hilos. En los cuatro casos de 5000 × 5000, la media de speedup fue 2,94 para 2 × 4 y 3,16 para 4 × 2; la diferencia describe este ensayo y no demuestra que 4 × 2 sea siempre superior.

14 Ventajas desventajas y limitaciones

La secuencial ofrece una referencia simple. OpenMP evita repartir la imagen entre espacios de memoria y requiere cuidar la independencia de las escrituras y las barreras entre etapas. MPI permite pensar el algoritmo para memoria distribuida, a cambio de halos y comunicaciones. La híbrida combina ambos esquemas, pero agrega sincronización y copias; una mayor cantidad de unidades no garantiza eficiencia alta. Se usó PPM P6 para evitar dependencias de decodificación en C++, con mayor tamaño de archivo que JPEG o PNG. Las entradas de benchmark son sintéticas y deterministas; una fotografía real de 1330 × 1002 se procesó aparte y sus cuatro salidas coincidieron en SHA-256. Las mediciones de tiempos cortos pueden fluctuar por la resolución del reloj y la actividad del sistema; hay tres repeticiones, sin intervalos de confianza.

El alcance experimental está limitado a un equipo Windows y a una implementación concreta de cada modelo. La medida secuencial y OpenMP abarca solo cómputo, mientras que MPI e híbrida incluyen distribución y reunión de datos; por ello el speedup compara el tiempo observado de cada programa, no kernels de cálculo idénticos. No se separaron tiempos de comunicación, creación de hilos, asignación de memoria ni entrada y salida. La repetición de tres corridas permite calcular medias y observar dispersión individual en mediciones.csv, pero no permite afirmar diferencias pequeñas con significación estadística.

15 Conclusiones

Las cuatro implementaciones compilaron y produjeron resultados equivalentes en toda la matriz exigida. Las pruebas adicionales compararon 260 salidas pequeñas píxel por píxel con una referencia independiente y comprobaron 32 rechazos de entradas, parámetros o destinos inválidos. En el hardware utilizado, OpenMP con 8 hilos lideró ocho casos y la híbrida 4 × 2 lideró cuatro. El rendimiento mejoró con más trabajo por imagen, pero la eficiencia permaneció por debajo de uno al aumentar las unidades. La distribución por filas con halos resolvió correctamente las dependencias espaciales sin artefactos en las fronteras de proceso.

16 Referencias bibliográficas

Guerrero, R. (2017). Procesamiento de imágenes [Apuntes de cátedra]. Archivo Apuntes.pdf.
UADER. (2026). Computación Avanzada: Práctico Final Integrador [Consigna de cátedra]. Archivo CA-Practico Final-2026.pdf.

Datos de respaldo: resultados/matriz_completa/mediciones.csv, resumen.csv y validacion.txt. Los seis gráficos se generaron exclusivamente desde resumen.csv.